자동 주행 및 장애물 회피 기술을 활용한 미로 통과 로봇

박상욱, 박민상, 우동식, 강성묵
대구가톨릭대학교 로봇공학과

A maze-passing robot using Automatic Driving and Obstacle avoidance technology

Park Sang Wook, Park Min Sang, Woo Dong Sik, Kang Sung Mook
Daegu Catholic University, Department of Robotics Engineering

Abstract - 본 논문에서 제안한 미로 통과 로봇은 자동 주행 및 장애물 회피 기술을 활용하여 주어진 미로 환경을 효율적으로 탐색하고 자동 주행 및 장애물을 회피하는 로봇이다. 로봇 공학에서 활용되는 센서 활용 알고리즘 설계 및 3D 프린팅을 종합적으로 적용하여 실제 문제 해결 능력을 향상하는 것을 목표로 한다. 이를 통해 자율 주행 로봇 기술의 기초를 익히고 로봇이 환경을 인식하고 반응하는 원리를 학습하는 것을 목표로 하였다.

1. 서 론

최근 자율주행 기술은 다양한 분야에서 응용되고 있으며, 이를 소형 로봇에 구현함으로써 알고리즘과 하드웨어 연동에 대한 이해를 높일 수 있다. 본 논문에서는 자율주행 로봇을 직접 제작하고, 수동 RC 제어 기능도 함께 제공하여 다양한 상황에서 운용 가능한 시스템을 구축하였다.

2. 본 론

2.1 시스템 설계 개요

제안된 로봇은 탱크 바퀴 기반의 주행 로봇으로, 2개의 DC 모터와 L298N 모터 드라이버를 통해 구동된다. 주행 안정성과 효율성을 높이기 위해 초음파 센서를 활용하여 전방 및 측면 장애물을 감지하고, 이를 바탕으로 가장 개방된 방향으로 주행하는 자율주행 알고리즘을 적용하였다. RC모드에서는 조종기를 이용해 직접 제어가 가능하며, 버튼을 통해 자율주행과 RC모드 전환이 손쉽게 이루어진다. 또한, 가감속 기반 속도 제어 알고리즘을 도입하여 급격한 속도 변화로 인한 모터 손상과 주행 불안정을 최소화하였다. 특히, 주행 시 차동 회전 최적화 설계를 통해 정밀한 회전 동작을 구현하였으며, 각 바퀴의 속도 차이를 효과적으로 제어하여 회전 반경을 최소화하였다. 제어 알고리즘은 아두이노를 기반으로 구현되었으며, 실시간 센서 데이터를 처리하여 즉각적인 제어 명령을 수행한다. 전원 시스템은 18650 리튬이온 충전지 3000mAh 3개를 병렬 구성하여 안정적인 전력 공급을 보장한다. 전체 시스템은 3D 프린팅을 활용하여 경량화 및 내구성을 확보하였다

2.1.1 차동 회전 최적화 설계

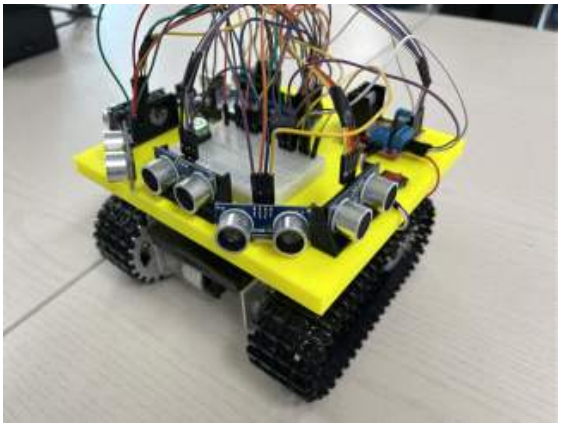
본 시스템은 조향 입력에 따라 좌우 바퀴의 속도를 독립적으로 제어하는 차동 회전 구조를 채택하였다. 이를 통해 협소한 공간에서도 정밀한 회전 및 선회를 구현하였으며, 회전 반대측 바퀴의 최소 속도를 보정하는 기능을 추가하여 한쪽 바퀴의 정지로 인한 불안정한 주행을 방지하였다. 이로써 곡선 주행 및 제자리 회전 시에도 부드럽고 안정적인 동작을 실현하였다.

2.2 가감속 기반 속도 제어 알고리즘 적용

갑작스러운 속도 변화로 인한 기계적 충격 및 차량 전복을 방지하기 위해, 가감속 알고리즘을 적용하였다. 사용자의 RC 입력을 즉시 모터에 반영하는 대신, 설정된 가감속 스텝만큼 점진적으로 속도를 변경하도록 하여, 보다 부드럽고 예측 가능한 주행이 가능하도록 설계하였다. 이로 인해 초보 사용자도 안정적으로 로봇을 조작할 수 있으며, 주행 중 급격한 출발·정지로 인한 시스템 손상을 최소화할 수 있다.

<표 1> 가감속 기반 속도 제어 알고리즘 구성 및 기대 효과

구성 요소	설명	기대 효과
속도 변화 제한 구간	일정 시간 또는 거리 내 속도 증가/감소 폭 제한	급가속·급감속 방지, 부드러운 주행 구현
목표 속도 도달 보정	실제 속도와 목표 속도의 차이를 지속적으로 계산 및 조정	목표 속도에 정확하고 안정적으로 도달
순간 급변 입력 필터링	사용자 또는 시스템 입력의 급격한 변화 감지 및 필터링 처리	입력으로 인한 제어 불안정 최소화



<그림 1> 시스템 실제 사진



<그림 2> 로봇 주행 사진

3. 결 론

본 논문에서는 탱크 바퀴 기반 이동 로봇을 제작하고, 가감속 기반 속도 제어 알고리즘과 차동 회전 최적화 설계를 통해 주행 성능을 향상시켰다. 실험 결과, 속도 변동이 부드러워지고, 회전 반경이 개선되어 협소한 공간에서도 안정적인 주행이 가능함을 확인하였다. 또한, 자율주행과 RC모드를 버튼을 통해 손쉽게 전환할 수 있어 사용자의 편의성이 높아졌다. 전체 시스템은 안정적인 하드웨어 구성과 효율적인 소프트웨어 제어를 통해 실용성을 확보하였다. 향후에는 보다 정밀한 위치 인식 기술을 추가하여 주행 정확도를 개선하고, 비정형 환경에서도 안정적인 자율주행이 가능하도록 알고리즘을 고도화할 예정이다. 또한, 추가 센서를 활용한 장애물 회피 기능 개선 및 비상 정지 시스템을 도입할 계획이다. 본 연구 결과는 소형 자율주행 로봇 및 탐색 로봇 분야에 실질적인 기여를 할 수 있을 것으로 기대된다. 궁극적으로는 산업 및 교육용 로봇 개발의 기초 자료로 활용될 수 있을 것이다.

[참 고 문 헌]

[1] 김현수, 박진우, "이동 로봇을 위한 속도 프로파일 기반 가감속 제어 알고리즘 연구", 한국로봇학회논문지, 제15권, 제2호, pp. 123-130, 2019.
[2] 이재훈, "차륜형 이동 로봇의 주행 안정성 향상을 위한 차동 구동 최적화 연구", 한국기계학회논문집, 제42권, 제5호, pp. 457-464, 2022.
[3] 김성민, "자율주행 로봇을 위한 속도 및 방향 제어 알고리즘 설계", 한양대학교 대학원 석사학위논문, 2021.
[4]S. Thrun, W. Burgard, D. Fox, Probabilistic Robotics, MIT Press, 2005.

[첨부-코드]

```
// 모터 제어 핀
int motor1Pin1 = 22;
int motor1Pin2 = 23;
int motor2Pin1 = 24;
int motor2Pin2 = 25;

// PWM 제어 핀 (ENA, ENB)
int motor1Enable = 5;
int motor2Enable = 6;

// 초음파 센서 핀
const int trigFront = 30;
const int echoFront = 31;
const int trigLeft = 32;
const int echoLeft = 33;
const int trigRight = 34;
const int echoRight = 35;
const int trigFLeft = 36;  // 추가된 좌측 대각선 센서
const int echoFLeft = 37;  // 추가된 좌측 대각선 센서
const int trigFRight = 38; // 추가된 우측 대각선 센서
const int echoFRight = 39; // 추가된 우측 대각선 센서

// 버튼 핀
const int buttonPin = 2;
bool autoMode = true;
bool lastButtonState = HIGH;
bool currentButtonState;

// RC 수신기 핀
const int throttlePin = 3;
const int steeringPin = 4;

// 현재 모터 속도 (가감속용)
int currentLeftSpeed = 0;
int currentRightSpeed = 0;

// 가감속 스텝 (속도 변할 때 한번에 바뀌는 최대값)
const int accelStep = 40;  // 가감속 스텝 키움

void setup() {
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(motor2Pin1, OUTPUT);
  pinMode(motor2Pin2, OUTPUT);
  pinMode(motor1Enable, OUTPUT);
  pinMode(motor2Enable, OUTPUT);

  pinMode(trigFront, OUTPUT);
  pinMode(echoFront, INPUT);
  pinMode(trigLeft, OUTPUT);
  pinMode(echoLeft, INPUT);
  pinMode(trigRight, OUTPUT);
  pinMode(echoRight, INPUT);
  pinMode(trigFLeft, OUTPUT);  // 추가된 센서 핀
  pinMode(echoFLeft, INPUT);   // 추가된 센서 핀
  pinMode(trigFRight, OUTPUT); // 추가된 센서 핀
  pinMode(echoFRight, INPUT);  // 추가된 센서 핀

  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(throttlePin, INPUT);
  pinMode(steeringPin, INPUT);

  Serial.begin(9600);
  Serial.println("시작 준비 완료!");
}

void loop() {
  checkModeButton();

  if (autoMode) {
    runAutonomousMode();
  } else {
    runRCMode();
  }
}

void checkModeButton() {
  currentButtonState = digitalRead(buttonPin);

  if (lastButtonState == HIGH && currentButtonState == LOW) {
    autoMode = !autoMode;
    Serial.print("모드 전환됨: ");
    Serial.println(autoMode ? "자율 주행 모드" : "RC 수동 모드");
    delay(500);
  }

  lastButtonState = currentButtonState;
}

void runAutonomousMode() {
  long front = measureDistance(trigFront, echoFront);
  long left = measureDistance(trigLeft, echoLeft);
  long right = measureDistance(trigRight, echoRight);
  long fLeft = measureDistance(trigFLeft, echoFLeft);  // 좌측 대각선 거리
  long fRight = measureDistance(trigFRight, echoFRight); // 우측 대각선 거리

  Serial.print("Front: ");
  Serial.print(front);
  Serial.print(" | Left: ");
  Serial.print(left);
  Serial.print(" | Right: ");
  Serial.print(right);
  Serial.print(" | FLeft: ");
  Serial.print(fLeft);
  Serial.print(" | FRight: ");
  Serial.println(fRight);

  // 1. 정면에 장애물이 없으면 계속 전진 (앞으로 20cm 이내에서 반응)
  if (front > 30) {
    moveForwardWithAlign();
  }
  // 2. 정면에 장애물이 있고, 좌측/우측에 장애물이 없으면 회전
  else {
    if (left > 30 && right <= 30) {  // 좀 더 멀리서 좌측 장애물 감지
      turnRight();
    } else if (right > 30 && left <= 30) {  // 좀 더 멀리서 우측 장애물 감지
      turnLeft();
    }
    // 3. 양측에 모두 장애물이 있을 때 대각선 센서 값 활용
    else if (left > 30 && right > 30) {
      // 대각선 센서가 장애물을 감지하면 회전 방향을 결정
      if (fLeft < 30 && fRight < 30) {  // 양쪽 대각선에 장애물
        if (front > 30) {
          moveForwardWithAlign();  // 장애물이 너무 많으면 전진
        } else {
          stopMotors();  // 이동 불가하면 정지
        }
```

```
      }
      // 좌측 대각선에 장애물이 있는 경우 우회전
      else if (fLeft < 30 && right > 30 && front > 30) {
        turnRight();  // 좌측 대각선에 장애물이 있으면 우회전
      }
      // 우측 대각선에 장애물이 있는 경우 좌회전
      else if (fRight < 30 && left > 30 && front > 30) {
        turnLeft();  // 우측 대각선에 장애물이 있으면 좌회전
      }
      // 대각선에는 장애물이 없고, 양측에 장애물이 있을 경우
      else {
        if (left > right) {
          turnLeft();
        } else {
          turnRight();
        }
      }
    }
    // 4. 정면에 장애물이 있으면 우회전 또는 좌회전
    else if (front <= 30) {  // 20cm로 멀리 설정
      if (left > right) {
        while (measureDistance(trigFront, echoFront) <= 30) {  // 20cm 이하일 때 회전
          turnRight();
          delay(100);
        }
      } else {
        while (measureDistance(trigFront, echoFront) <= 30) {  // 20cm 이하일 때 회전
          turnLeft();
          delay(100);
        }
      }
      moveForwardWithAlign();
    }
    // 5. 장애물이 없으면 전진
    else {
      stopMotors();
    }
  }
}




void runRCMode() {
  static int lastThrottle = 1400;
  static int lastSteering = 1400;

  int throttleRaw = pulseIn(throttlePin, HIGH, 50000);
  int steeringRaw = pulseIn(steeringPin, HIGH, 50000);

  if (throttleRaw > 900 && throttleRaw < 1800) lastThrottle = throttleRaw;
  if (steeringRaw > 1000 && steeringRaw < 1800) lastSteering = steeringRaw;

  int throttleVal = map(lastThrottle, 1000, 1800, -255, 255);
  int steeringVal = map(lastSteering, 1000, 1800, -255, 255);

  if (abs(throttleVal) < 30) throttleVal = 0;
  if (abs(steeringVal) < 30) steeringVal = 0;

  int targetLeftSpeed = throttleVal + steeringVal;
  int targetRightSpeed = throttleVal - steeringVal;

  int minTurnSpeed = 60;

  if (steeringVal < 0) {  // 왼쪽 회전 시 오른쪽 바퀴 최소 속도 보정
    if (abs(targetRightSpeed) < minTurnSpeed) {
      targetRightSpeed = (targetRightSpeed > 0) ? minTurnSpeed : -minTurnSpeed;
    }
  }
  if (steeringVal > 0) {  // 오른쪽 회전 시 왼쪽 바퀴 최소 속도 보정
    if (abs(targetLeftSpeed) < minTurnSpeed) {
      targetLeftSpeed = (targetLeftSpeed > 0) ? minTurnSpeed : -minTurnSpeed;
    }
  }

  // 가감속 적용
  currentLeftSpeed = gradualChange(currentLeftSpeed, targetLeftSpeed, accelStep);
  currentRightSpeed = gradualChange(currentRightSpeed, targetRightSpeed, accelStep);

  Serial.print("Throttle: ");
  Serial.print(throttleVal);
  Serial.print(" | Steering: ");
  Serial.print(steeringVal);
  Serial.print(" | LeftSpeed: ");
  Serial.print(currentLeftSpeed);
  Serial.print(" | RightSpeed: ");
  Serial.println(currentRightSpeed);

  setMotorSpeed(currentLeftSpeed, currentRightSpeed);
}

long measureDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 25000);
  if (duration == 0) return 999;
  return duration * 0.0344 / 2;
}

void setMotorSpeed(int left, int right) {
  left = constrain(left, -255, 255);
  right = constrain(right, -255, 255);

  if (left == 0) {
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
  } else if (left > 0) {
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
  } else {
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, HIGH);
    left = -left;
  }

  if (right == 0) {
    digitalWrite(motor2Pin1, LOW);
    digitalWrite(motor2Pin2, LOW);
  } else if (right > 0) {
    digitalWrite(motor2Pin1, HIGH);
    digitalWrite(motor2Pin2, LOW);
  } else {
    digitalWrite(motor2Pin1, LOW);
    digitalWrite(motor2Pin2, HIGH);
    right = -right;
  }

  analogWrite(motor1Enable, left);
  analogWrite(motor2Enable, right);
}

void moveForward() {
  setMotorSpeed(150, 140);
```

```
}

void turnLeft() {
  stopMotors();
  delay(500);
  setMotorSpeed(-160, 150);
  delay(500);  // 회전 시간 줄이기
  stopMotors();
}

void turnRight() {
  stopMotors();
  delay(500);
  setMotorSpeed(160, -150);
  delay(500);  // 회전 시간 줄이기
  stopMotors();
}


void stopMotors() {
  analogWrite(motor1Enable, 0);
  analogWrite(motor2Enable, 0);
}

void alignToCenter() {
  long left = measureDistance(trigLeft, echoLeft);
  long right = measureDistance(trigRight, echoRight);
  long diff = left - right;

  if (abs(diff) > 5) {
    if (diff > 0) {
      setMotorSpeed(80, -80);
      delay(120);
    } else {
      setMotorSpeed(-80, 80);
      delay(120);
    }
    stopMotors();
  }
}

void moveForwardWithAlign() {
  alignToCenter();
  setMotorSpeed(150, 140);
  delay(100);  // delay 줄임
  stopMotors();
}

// 가감속 함수 (현재 속도를 목표 속도에 최대 step만큼 점진적 변경)
int gradualChange(int current, int target, int step) {
  if (current < target) {
    current += step;
    if (current > target) current = target;
  } else if (current > target) {
    current -= step;
    if (current < target) current = target;
  }
  return current;
}
```